

Key Management

Introduction

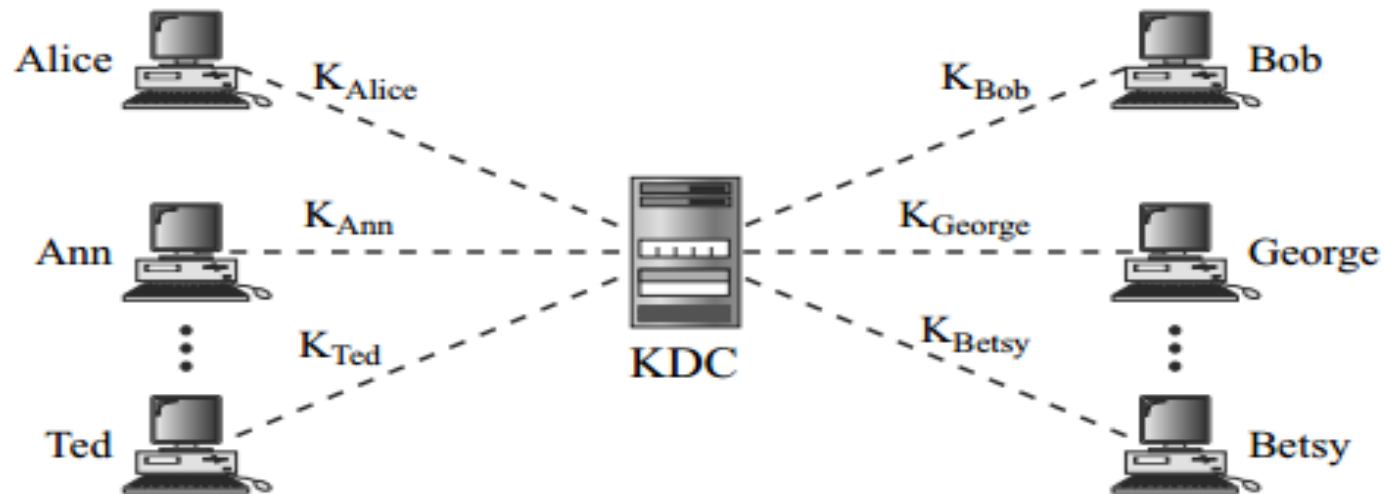
- **Symmetric-key cryptography** is more efficient than asymmetric-key cryptography for enciphering large messages
 - needs a shared secret key between two parties.
 - Is a **N^2 problem** as number of required keys for N entities is N^2 .
 - distribution of secret keys is another problem

Solution

use of a trusted third party, referred to as a **key-distribution center (KDC)**.

Key-Distribution Center(KDC)

- To reduce the number of keys, each person establishes a shared secret key with the KDC

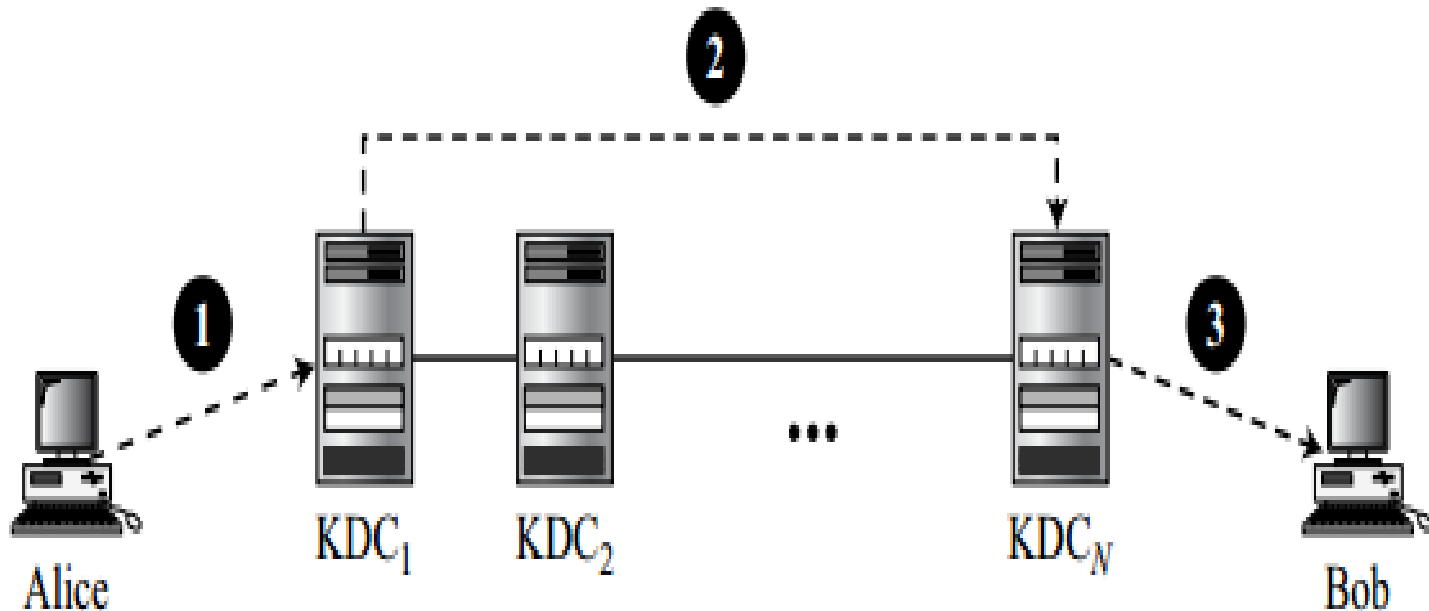


Key-Distribution Center(KDC)

- How Alice can send a confidential message to Bob?
 - Alice sends a request to the KDC stating that she needs a session (temporary) secret key between herself and Bob.
 - The KDC informs Bob about Alice's request.
 - If Bob agrees, a session key is created between the two.

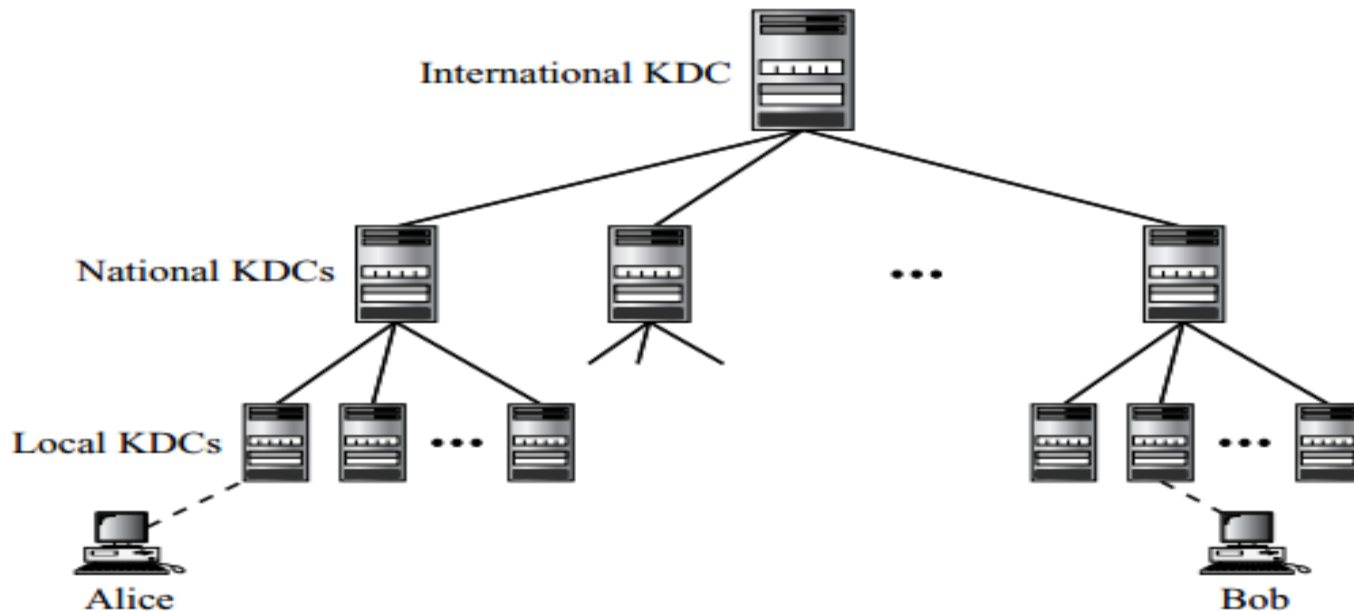
Multiple KDCs Structure

- Two types
 - Flat multiple KDCs
 - Each domain can have one or more KDCs



Multiple KDCs Structure

- Hierarchical Multiple KDCs
 - one or more KDCs at the top of the hierarchy
 - there can be local KDCs, national KDCs, and international KDCs.



Session Keys

- A KDC creates a secret key for each member.
- This secret key can be used only between the member and the KDC, **not between two members.**
- A session symmetric key between two parties is used only once.
- After communication is terminated, the session key is no longer useful.

Session Keys creation by KDC

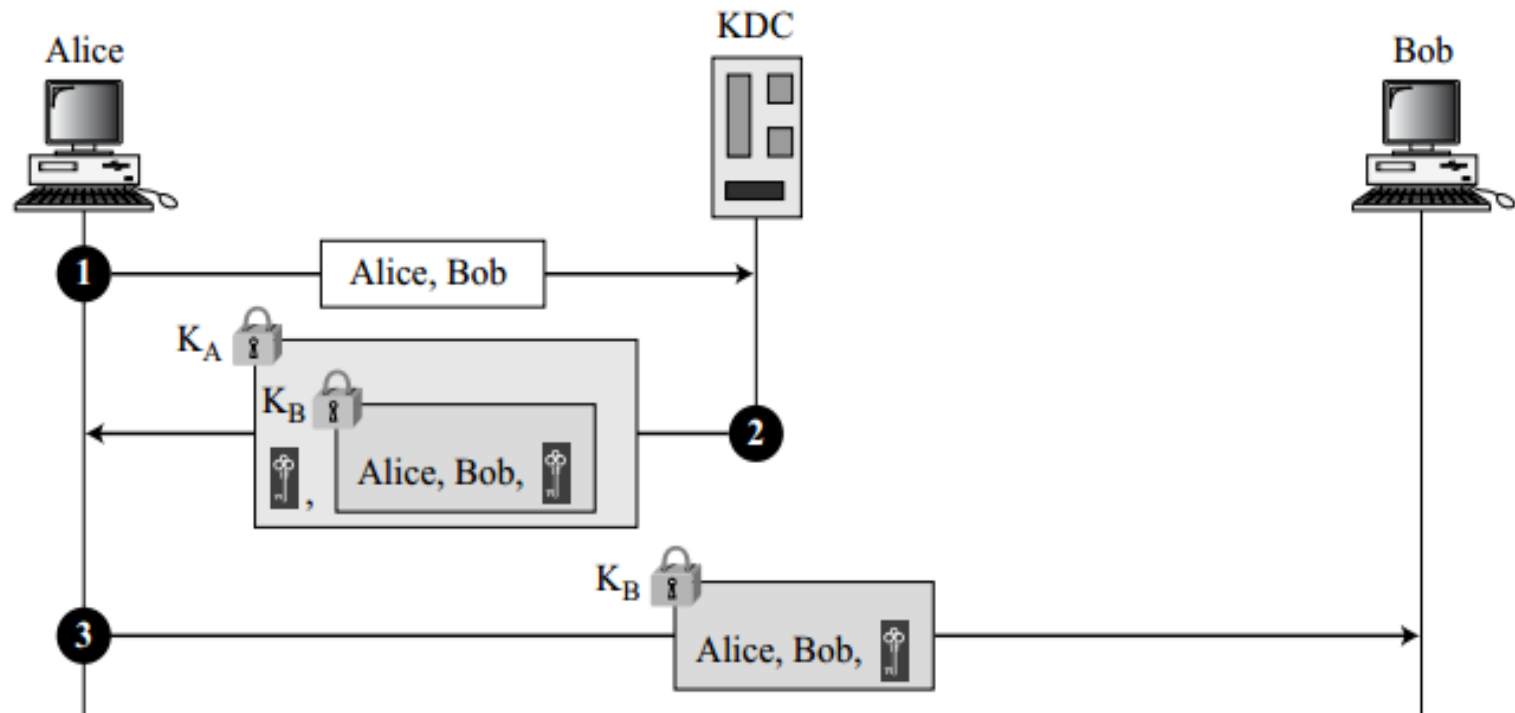
- First approach

K_A  Encrypted with Alice-KDC secret key

 Session key between Alice and Bob

K_B  Encrypted with Bob-KDC secret key

KDC: Key-distribution center



Session Keys creation by KDC

- Alice sends a plaintext message (IDs of Alice and Bob) to the KDC to obtain a symmetric session key between Bob and herself.
- The KDC receives the message and creates a **ticket** which contains the identities of Alice and Bob and the session key (K_{AB})
- The ticket is encrypted using Bob's key (K_B) and sent to Alice.
- Alice receives the message, decrypts it, and extracts the session key.
- Alice sends the ticket to Bob.
- Bob is authenticated to the KDC, also authenticated to Alice, so, only Bob can open the ticket

Session Keys creation by KDC

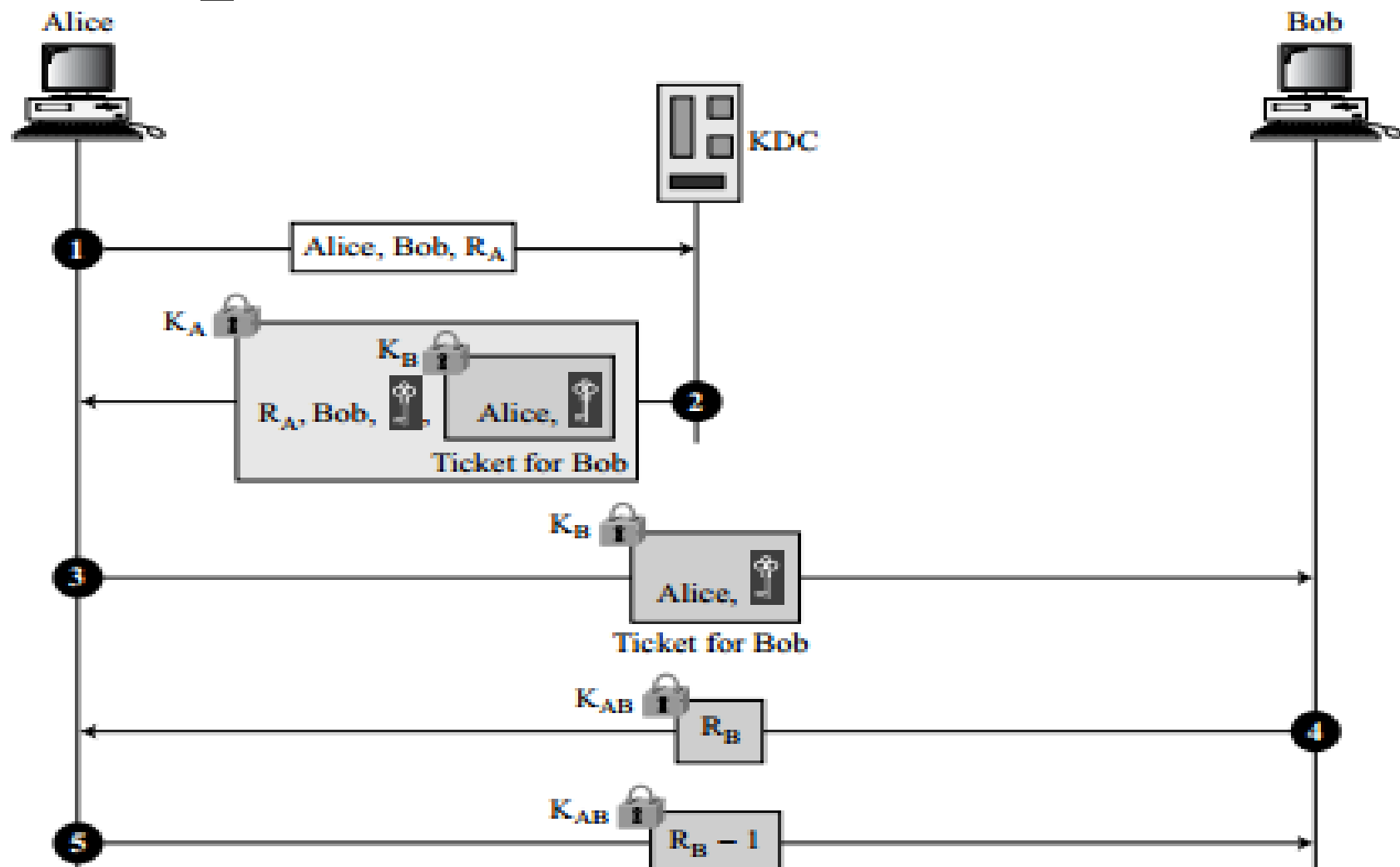
- Second Approach: Needham-Schroeder Protocol
 - uses multiple challenge-response interactions between parties to achieve a flawless protocol.
 - uses two nonces: RA and RB.

- K_A  Encrypted with Alice-KDC secret key
- K_B  Encrypted with Bob-KDC secret key
- K_{AB}  Encrypted with Alice-Bob session key
-  Session key between Alice and Bob

KDC: Key-distribution center

R_A : Alice's nonce

R_B : Bob's nonce



The following briefly describes the steps.

1. Alice sends a message to Bob that includes a common nonce, R , the identities of Alice and Bob, and a ticket for KDC that includes Alice's nonce R_A (a challenge for the KDC to use), a copy of the common nonce, R , and the identities of Alice and Bob.
2. Bob creates the same type of ticket, but with his own nonce R_B . Both tickets are sent to the KDC.
3. The KDC creates a message that contains R , the common nonce, a ticket for Alice and a ticket for Bob; the message is sent to Bob. The tickets contain the corresponding nonce, R_A or R_B , and the session key, K_{AB} .
4. Bob sends Alice her ticket.
5. Alice sends a short message encrypted with her session key K_{AB} to show that she has the session key.

Kerberos

Introduction to Kerberos

- The Kerberos Authentication Service, developed at MIT, provides a trusted third-party authentication to verify users' identity
- It uses symmetric key encryption techniques and a key distribution centre
 - it is an add-system that can be used with existing network.

Introduction to Kerberos

- Versions 1-3 were for internal use
- versions 4 and 5 are being used externally [?]
- Version 4 has a larger installed base, is simpler, and has better performance, but works only with TCP/IP networks [?]
- Version 5 developed in mid 90's (RFC-1510) corrects some of the security deficiencies of Version 4

Objective

- To provide a trusted third-party service (based on the Needham/Schroeder authentication protocol)
- can perform authentication between any pair of entities in TCP/IP networks [?]
- primarily used to authenticate user-at workstation to server [?]
- Authentication is two-way
- [?] Not meant for high risk operations (e.g., bank transactions, classified government data, student grades)

Kerberos Model

- Network consists of **clients and servers**
 - Clients may be users, or programs or processes that can download files, send messages or access databases
- Kerberos keeps a database of clients and servers with a secret key for each one (selected at the time of registration)
 - $O(n+m)$ keyspace, instead of $O(nm)$ keyspace with n clients and m servers

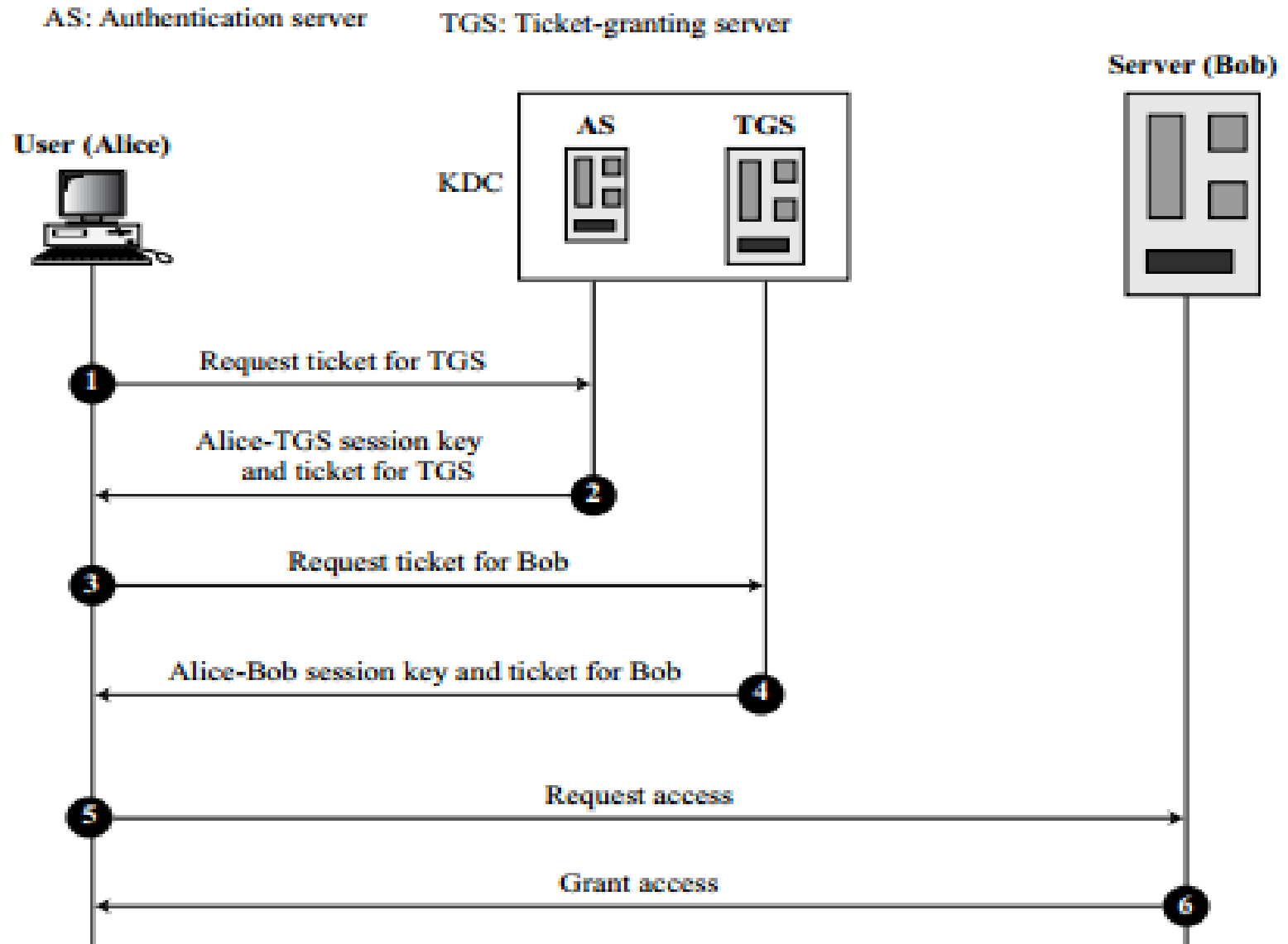
Kerberos Model

- Kerberos provides **authentication** of one entity to another and issues **session key**
- A **ticket** is an unforgeable, non replayable, authenticated object.
 - It is an encrypted data structure naming a user and a service that the user is allowed to obtain.
 - also contains a time value and some control information.
- Kerberos Issues **tickets** for access rights
 - Temporary rights issued by authentication server
 - Tickets time-stamped to reduce replay attacks

Kerberos servers

- Three servers are involved in the Kerberos protocol:
 - an authentication server (AS)
 - a ticket-granting server (TGS)
 - a real (data) server that provides services to others.

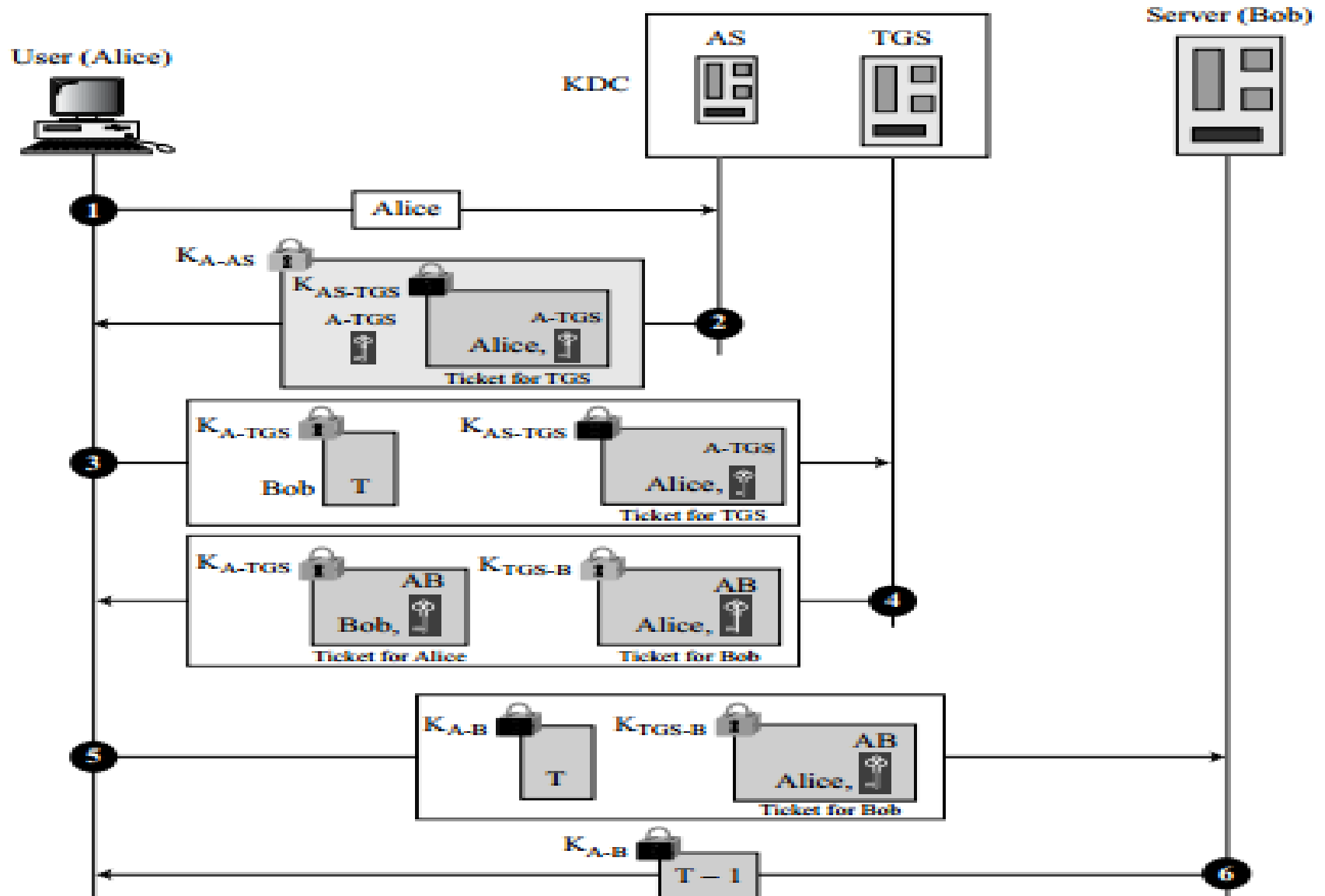
Kerberos servers



Kerberos servers

- The **authentication server (AS)** is the KDC in the Kerberos protocol.
 - Each user registers with the AS and is granted a user identity and a password.
 - The AS verifies the user, issues a session key to be used between client and the TGS, and sends a ticket for the TGS.
- The **ticket-granting server (TGS)** issues a ticket for the real server
 - provides the session key (KAB) between client and real server.
- The **real server** provides services for the user/client

Kerberos Operation



- K_{A-AS}  Encrypted with Alice-AS key
- K_{TGS-B}  Encrypted with TGS-Bob key
- K_{A-TGS}  Encrypted with AS-TGS key
- K_{AS-TGS}  Encrypted with Alice-TGS session key
- K_{A-B}  Encrypted with Alice-Bob session key

A-TGS

 Alice-TGS session key

AB

 Alice-Bob session key

KDC: Key-distribution center

AS: Authentication server

TGS: Ticket-granting server

T: Timestamp (nonce)

Kerberos Version 5

- Version 5 has a longer ticket lifetime.
- allows tickets to be renewed.
- can accept any symmetric-key algorithm.
- has more overhead than version 4.

Advantages

- Kerberos was designed to withstand attacks in distributed environment.
 - No password communicated on the network
 - Cryptographic protection (against spoofing)
 - Time stamps to prevent reply attacks
 - Limited period of validity
 - Each ticket is issued for a limited period of time

Drawbacks of Kerberos

- Single point of failure
 - requires continuous availability of the central server.
When the Kerberos server is down, no one can log on
(Solution: multiple Kerberos servers)
- Kerberos requires the clocks of the involved hosts to be synchronized.
- Since all authentication is controlled by a centralized KDC, compromise of this authentication infrastructure will allow an attacker to impersonate any user.

Symmetric Key Agreement

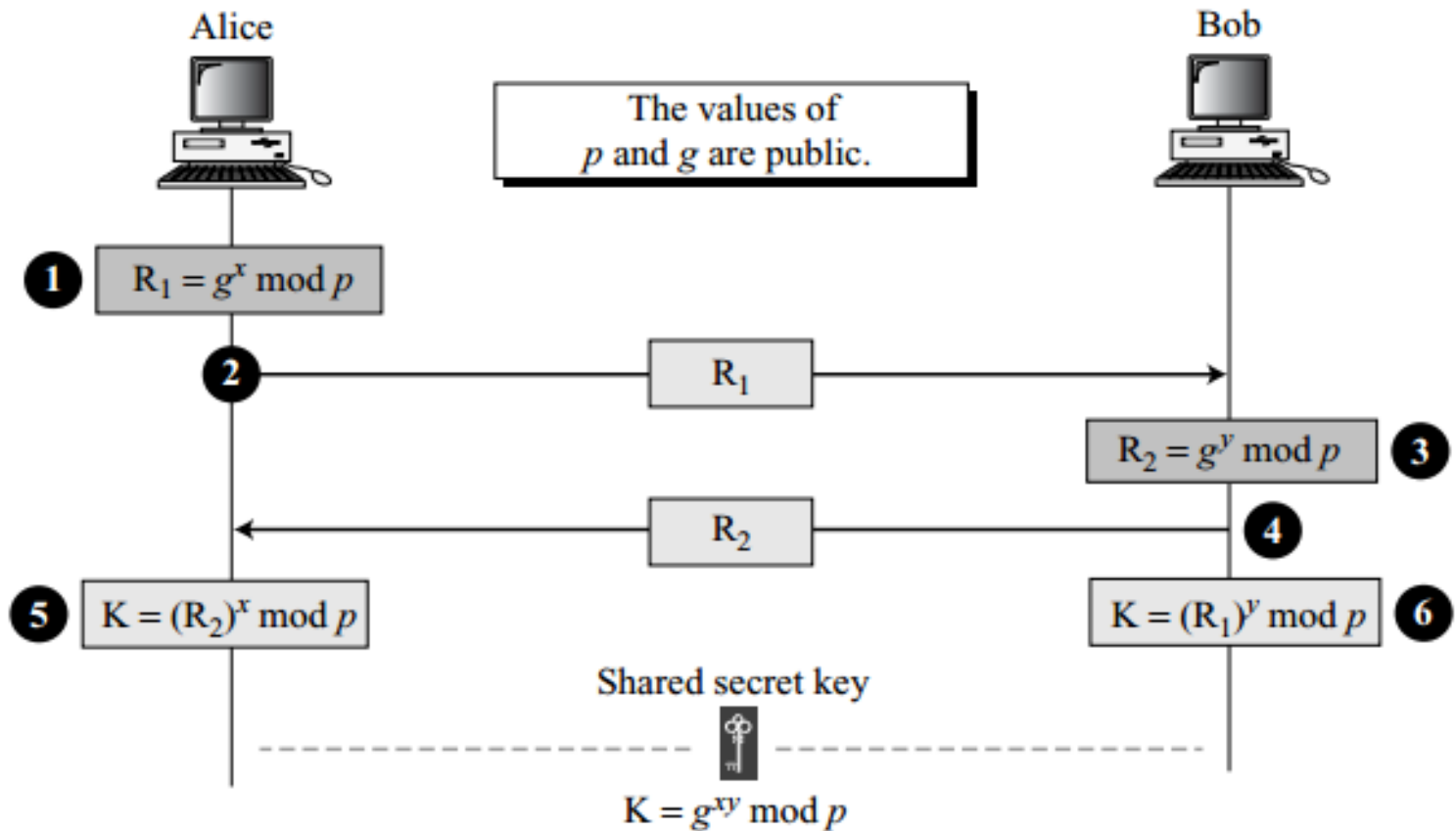
Symmetric Key Agreement

- It involves session key generation without involving KDC
- Two common methods of Symmetric Key Agreement are:
 - Diffie-Hellman Key Agreement
 - Station-to-Station Key Agreement

Diffie-Hellman Key Agreement

- Two parties create a symmetric session key without the need of a KDC.
- Before establishing a symmetric key, the two parties need to choose two numbers p and g .
 - p is a large prime number (1024 bits)
 - g is a generator of order $p - 1$ in the group $\langle \mathbb{Z}_p^*, \times \rangle$.

Diffie-Hellman method



$$K = (g^x \bmod p)^y \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$$

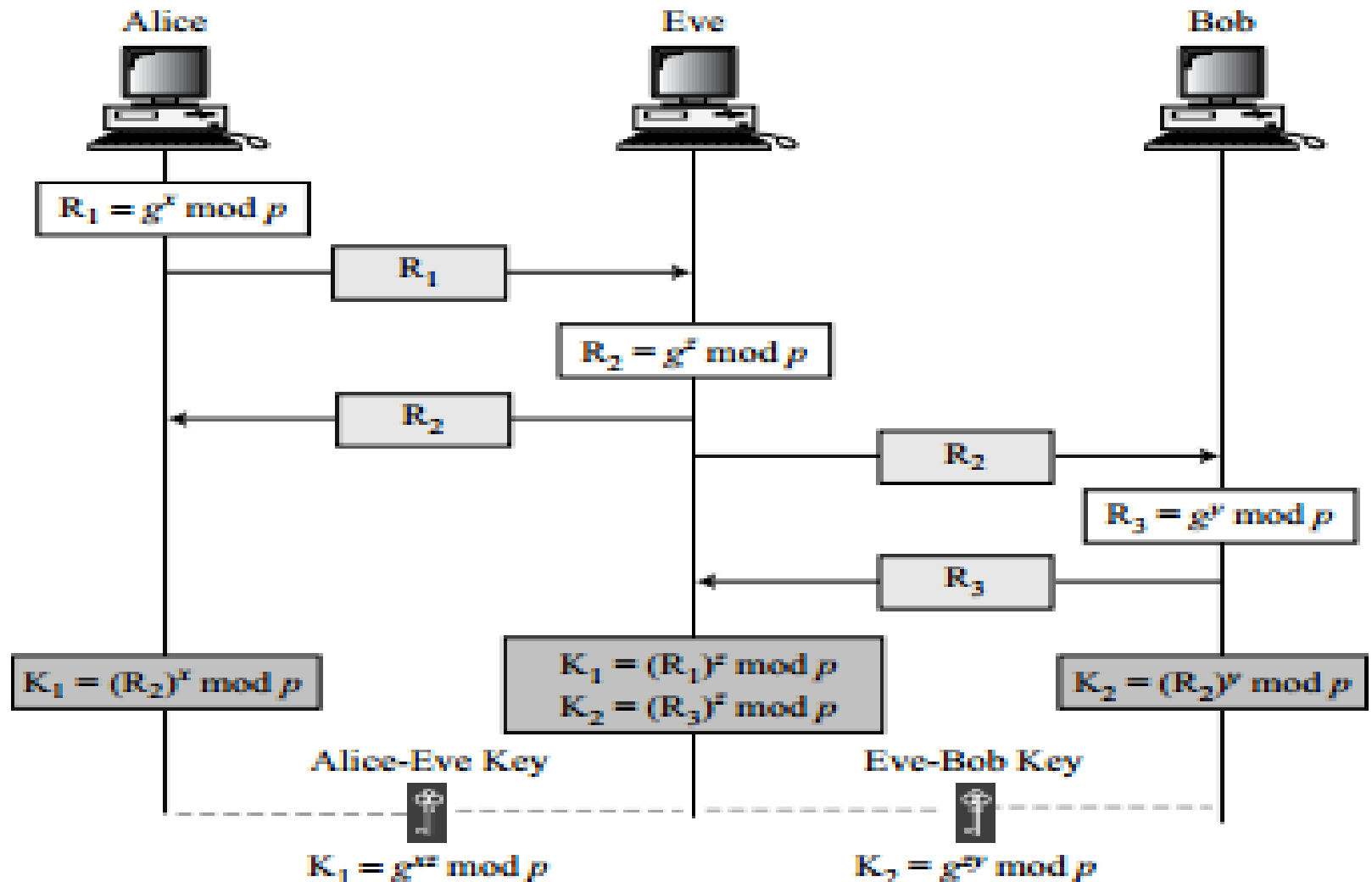
Security of Diffie-Hellman key exchange

- The Diffie-Hellman key exchange is susceptible to two attacks:
 - discrete logarithm attack
 - man-in-the-middle attack

Discrete Logarithm Attack

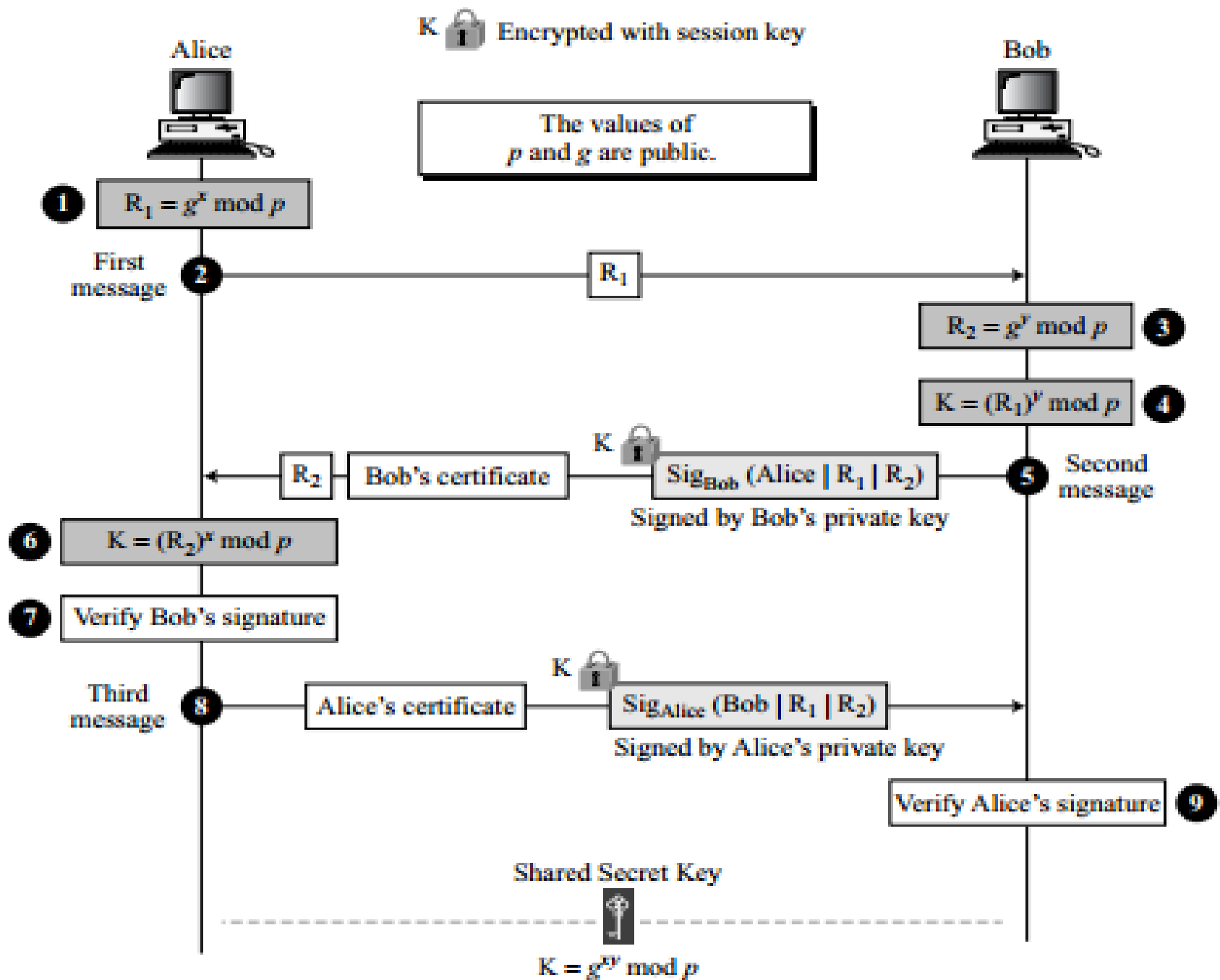
- The security of the DH key exchange is based on the difficulty of the discrete logarithm problem.
- To make Diffie-Hellman safe from the discrete logarithm attack, the participants must destroy x and y after they have calculated the symmetric key
 - values of x and y must be used only once

Man-in-the-Middle Attack in DH Key Exchange



Station-to-Station Key Agreement

- Based on Diffie-Hellman protocol.
- It uses **digital signatures** with public-key **certificates** to establish a session key between two participants
-



- The station-to-station protocol prevents man-in-the-middle attacks.

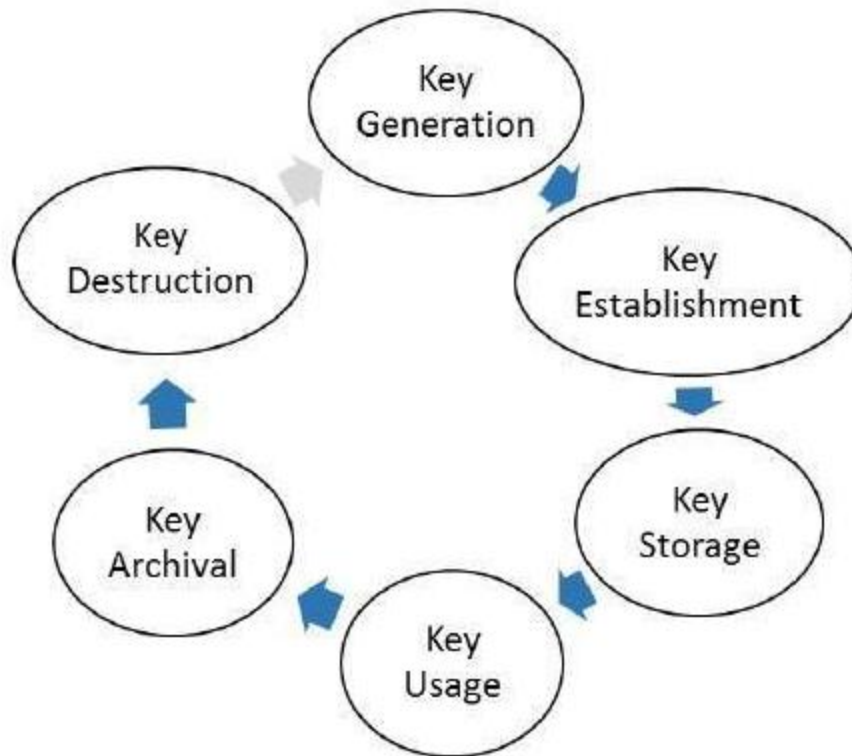
Public Key Management

Addressing the limitations of private-key encryption

- Public-key vastly reduces the need to store many different secret keys.
 - Even if a large number of pairs want to communicate secretly, each party needs store only one key: her own.
- Public-key allows **key distribution** to be done over public channels.
 - Initial deployment and system maintenance is simplified.

Key Management

- Refers to the secure administration of cryptographic keys(Key management deals with entire **key lifecycle**)



Key Management

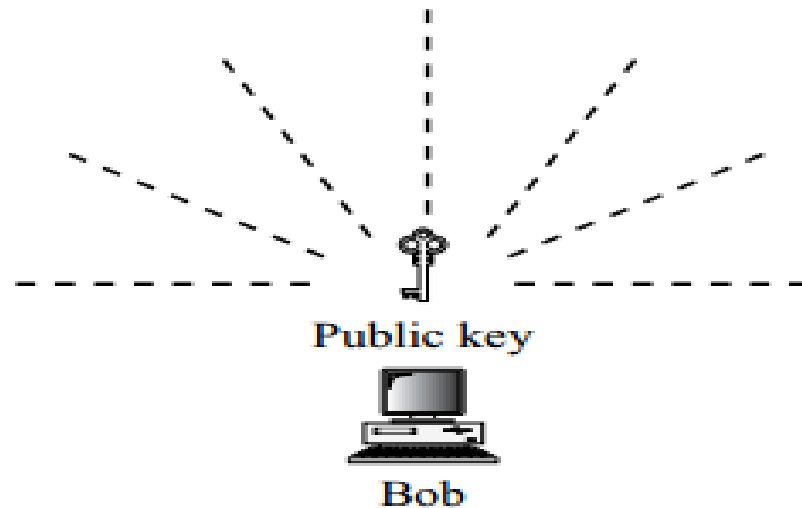
- Two specific requirements of key management for public key cryptography.
 - Secrecy of private keys
 - Assurance of public keys
 - can be achieved through the **public-key infrastructure (PKI)**, a key management systems for supporting public-key cryptography.

Announcing a public key

- Four methods:
 - Announce public keys publicly
 - Announce public keys Using Trusted Center
 - Announce public keys Using Controlled Trusted Center
 - Announce public keys Using Certification Authority (CA)

Announcing a public key

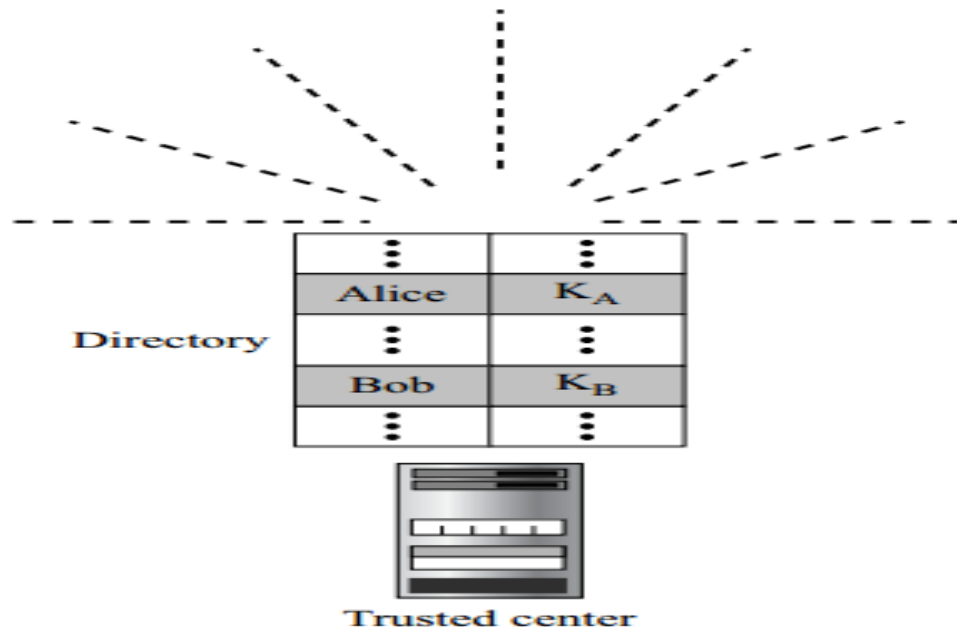
- **Announce public keys publicly**
 - naive approach
 - is not secure; subject to forgery.



Announcing a public key

- Using Trusted Center

- trusted center retain a directory of public keys which is dynamically updated
- each user register in the center and prove his or her identity.
- The directory can be publicly advertised by the trusted center.



Announcing a public key

- Using Controlled Trusted Center
 - there are added controls on the distribution of the public key.
 - public-key announcements can include a timestamp and be signed by an authority

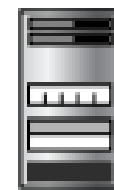


Alice



Bob's public key

T: Timestamp



Trusted center

Directory

⋮	⋮
Alice	K_A
⋮	⋮
Bob	K_B
⋮	⋮

Need Bob's key, T

Need Bob's key, T,



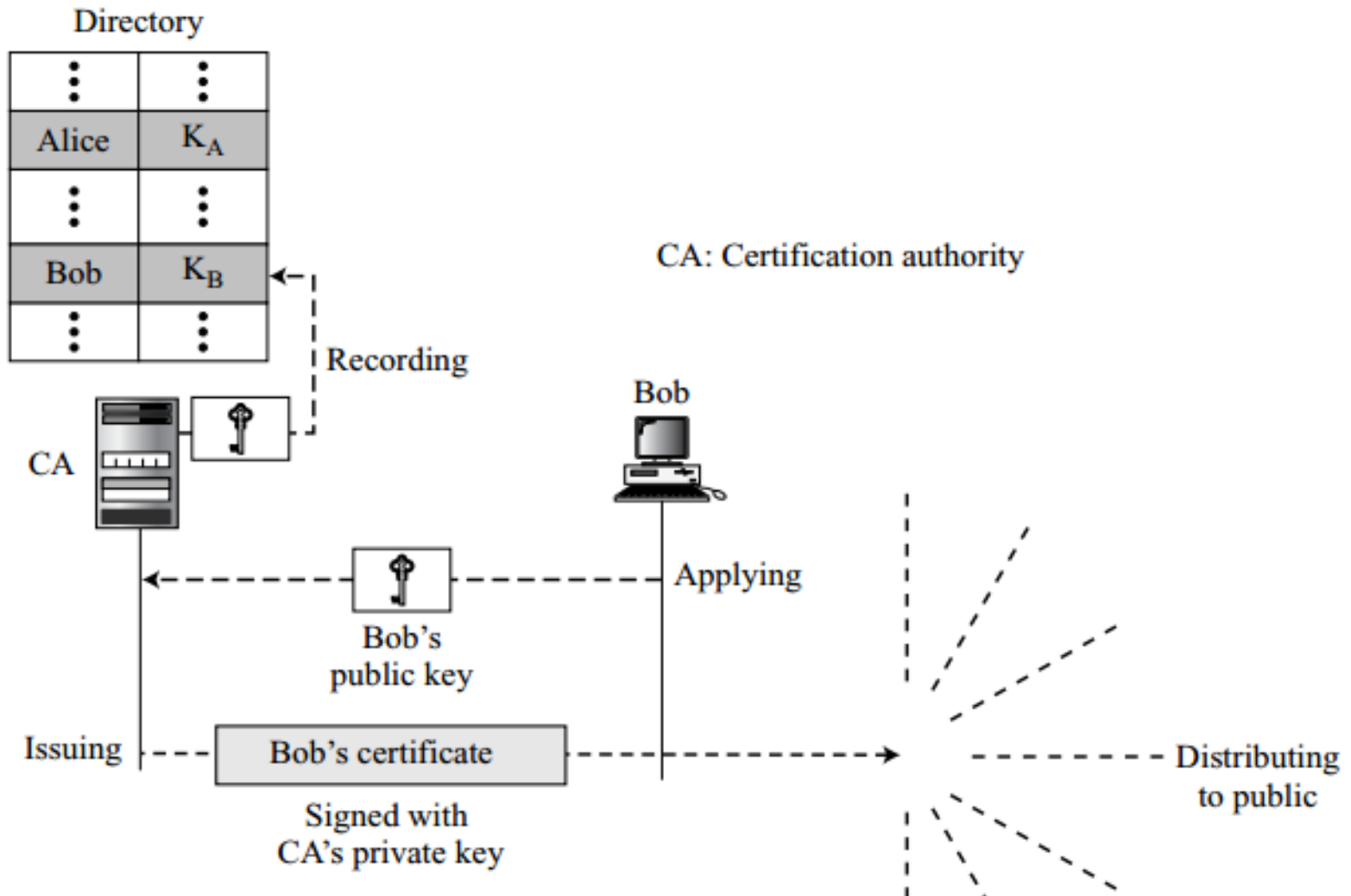
$\text{Sig}_{\text{center}}(T)$

Signed by center's
private key

Announcing a public key

- Using Certification Authority
 - use certificates authenticated by an authority (public-key certificates)
 - a certification authority (CA), a federal or state organization that binds a public key to an entity and issues a certificate.

Certification Authority



Key Functions of CA

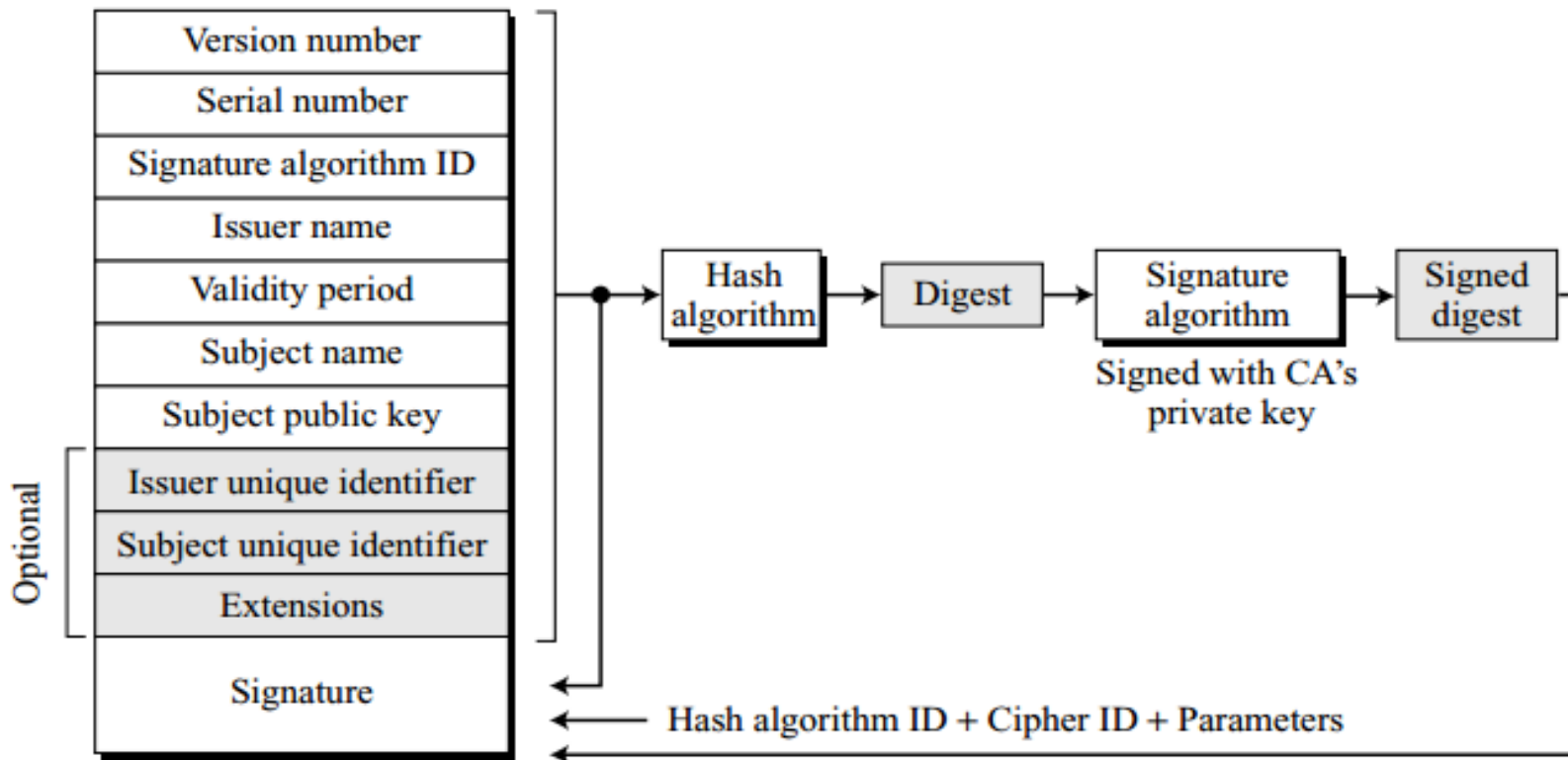
- Generating key pairs
 - The CA may generate a key pair independently or jointly with the client.
- Issuing digital certificates
- Publishing Certificates
- Verifying Certificates
- Revocation of Certificates

Digital Certificates

- Digital certificates are based on the ITU standard X.509
- Public key pertaining to the user client is stored in digital certificates by The Certification Authority (CA) along with other relevant information such as client information, expiration date, usage, issuer
- CA digitally signs this entire information and includes digital signature in the certificate.

X.509

- X.509 is a way to describe the certificate in a structured way.
- X.509 certificate format:



Certificate Renewal

- Each certificate has a period of validity.
- CA issues a new certificate before the old one expires

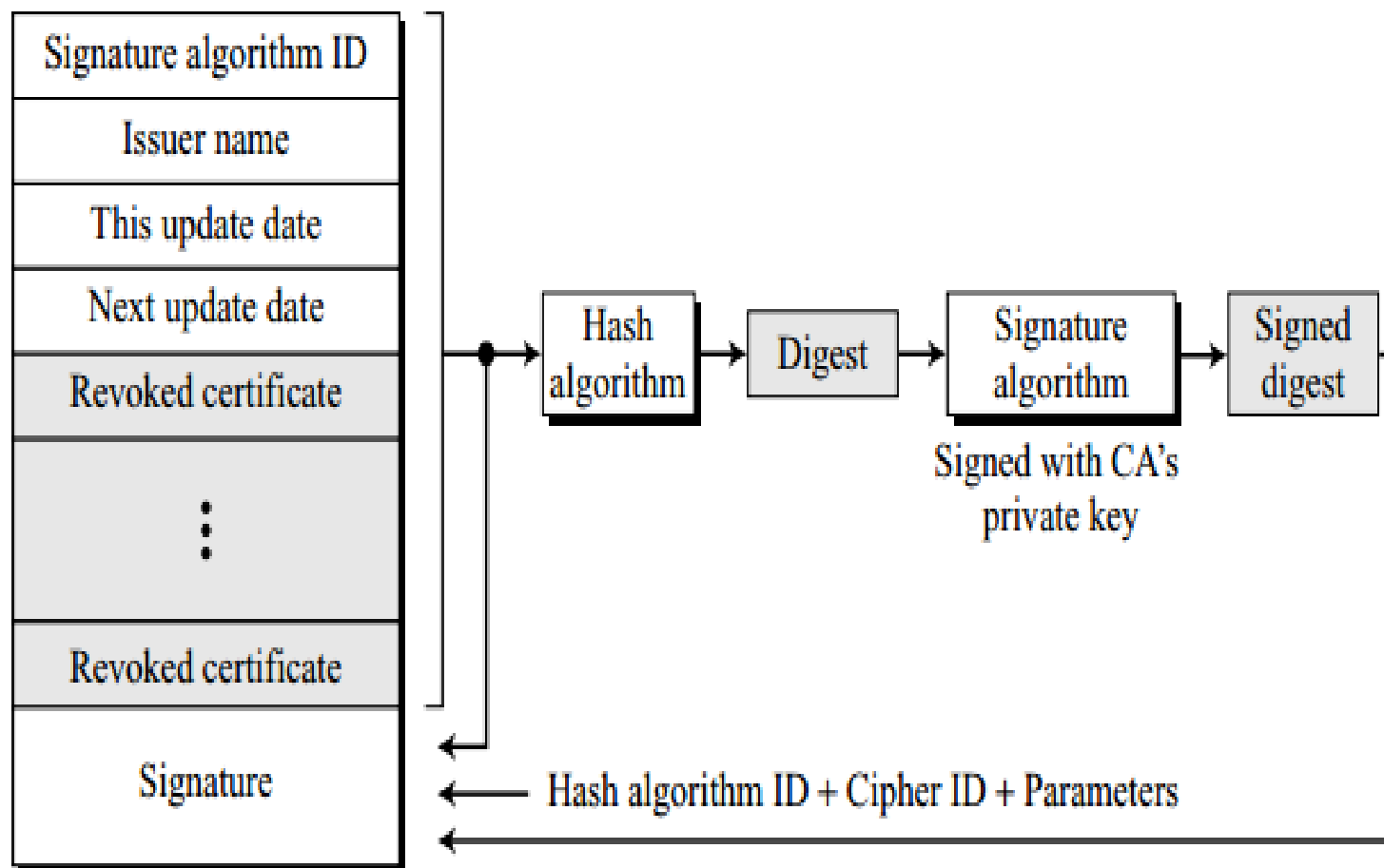
Certificate Revocation

- In some cases a certificate must be revoked before its expiration.
 - The user's private key corresponding to the public key listed in the certificate) might have been comprised.
 - The CA is no longer willing to certify the user.
 - The CA's private key, which can verify certificates, may have been compromised.

Certificate Revocation List

- The revocation is done by periodically issuing a certificate revocation list (CRL)
- CRL contains all revoked certificates that are not expired on the date the CRL is issued.
-

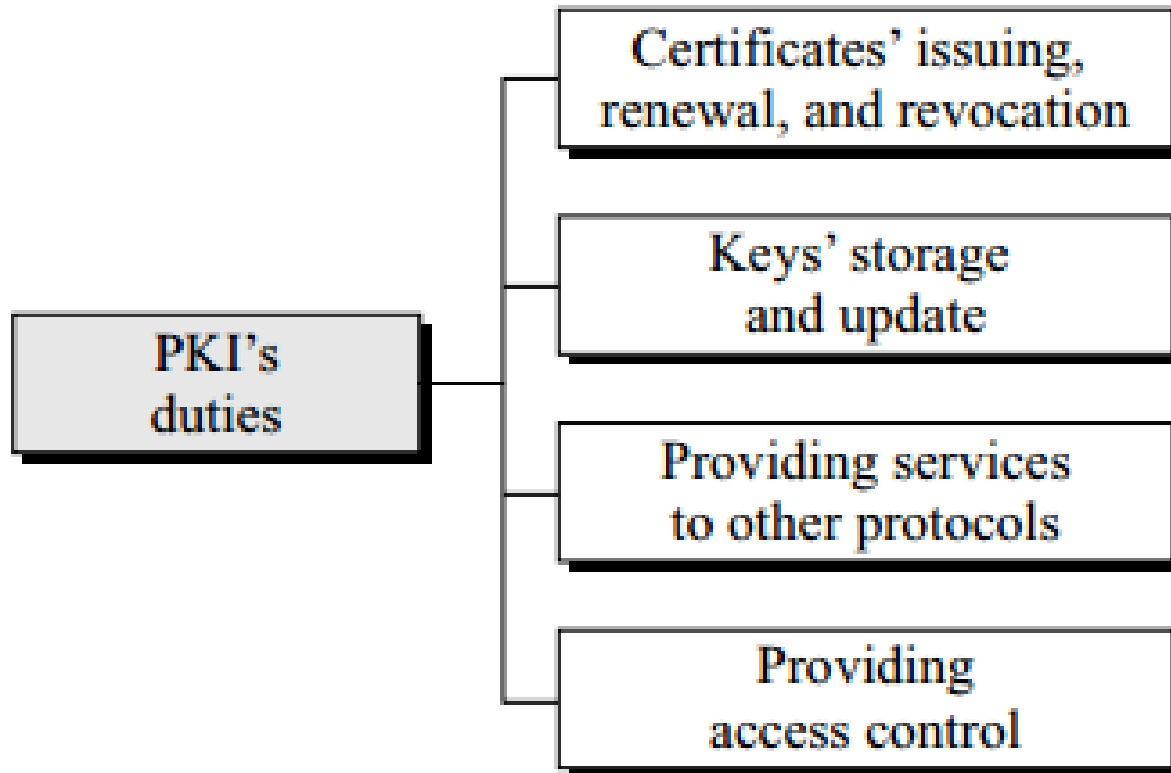
Certificate revocation format



Public-Key Infrastructures (PKI)

- It is a model for creating, distributing, and revoking certificates based on the X.509.
- The Internet Engineering Task Force has created the Public-Key Infrastructure [X.509 \(PKIX\)](#).

Some duties of a PKI

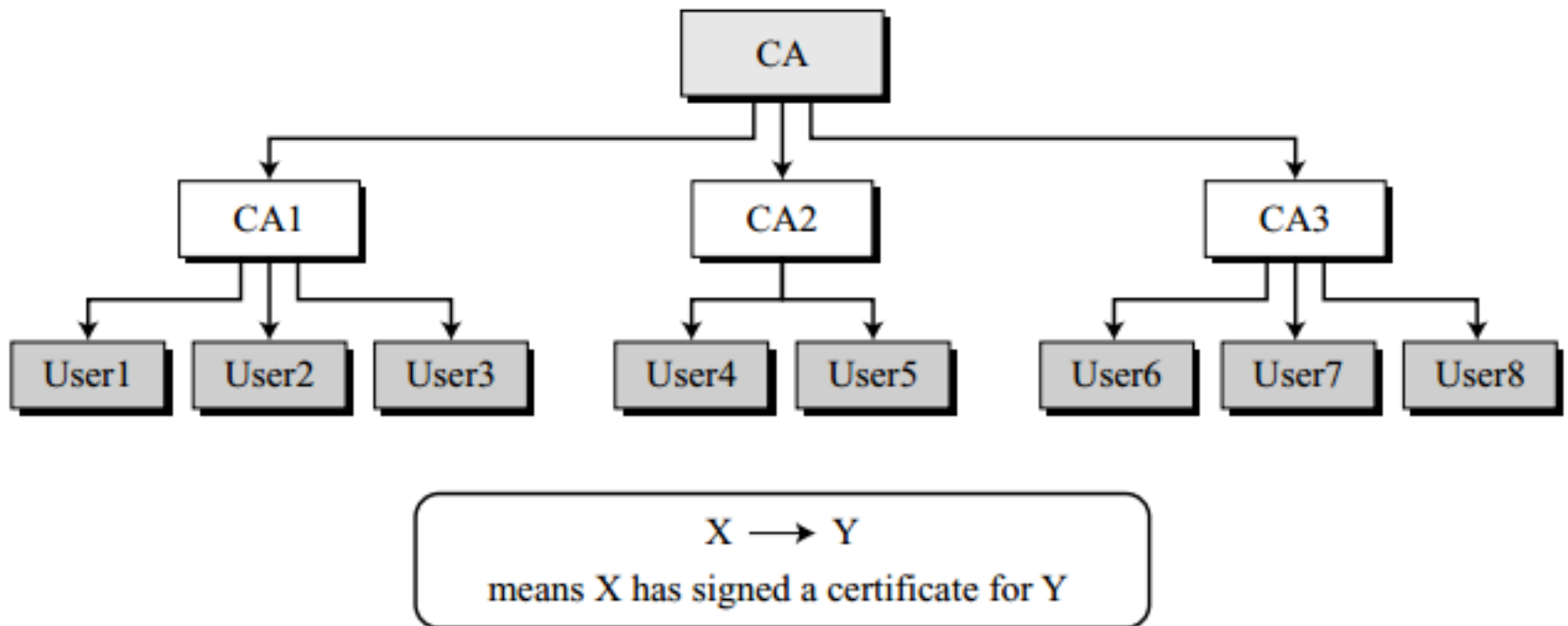


Trust Model

- The trust model defines rules that specify how a user can verify a certificate received from a CA.
- Two models are used:
 - PKI hierarchical model
 - PKI Mesh model

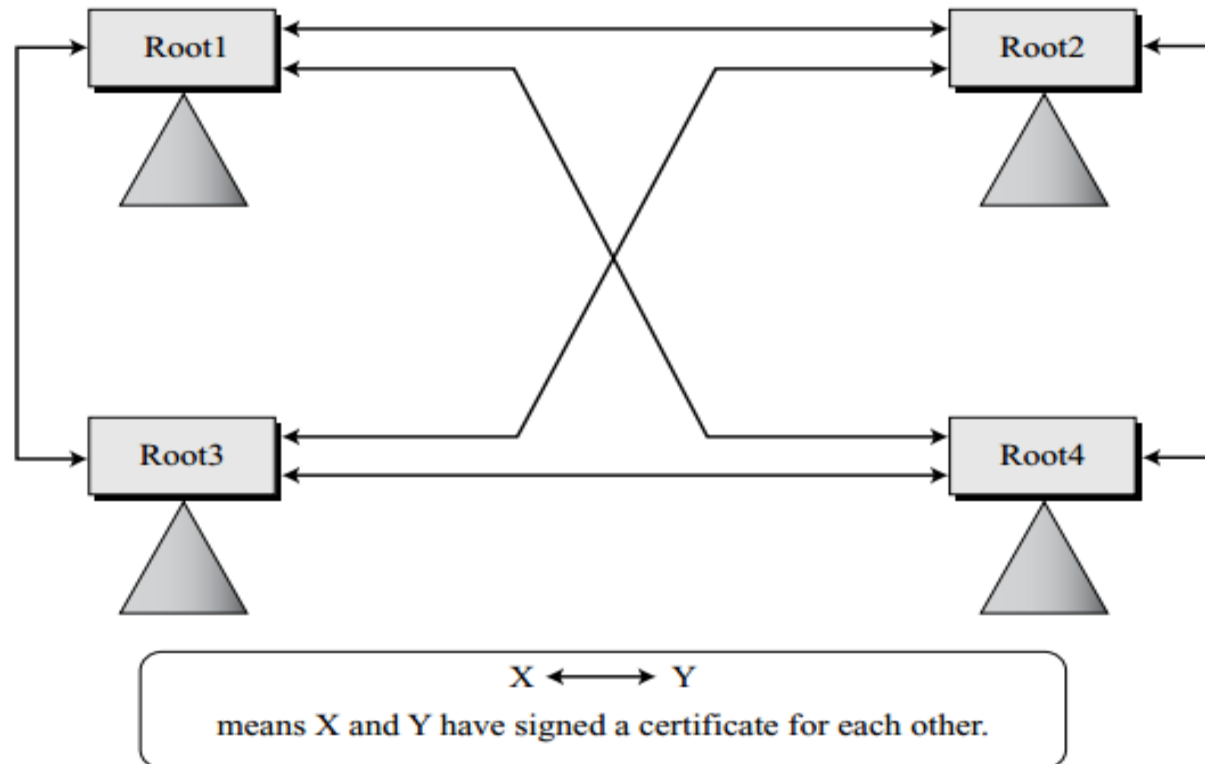
PKI hierarchical model

- It has a tree-type structure with a root CA.
- The root CA has a self-signed, self-issued certificate



PKI Mesh model

- Each root CA is connected to every other root CA
- Each root certifies all other roots, which means there are $N(N - 1)$ certificates.

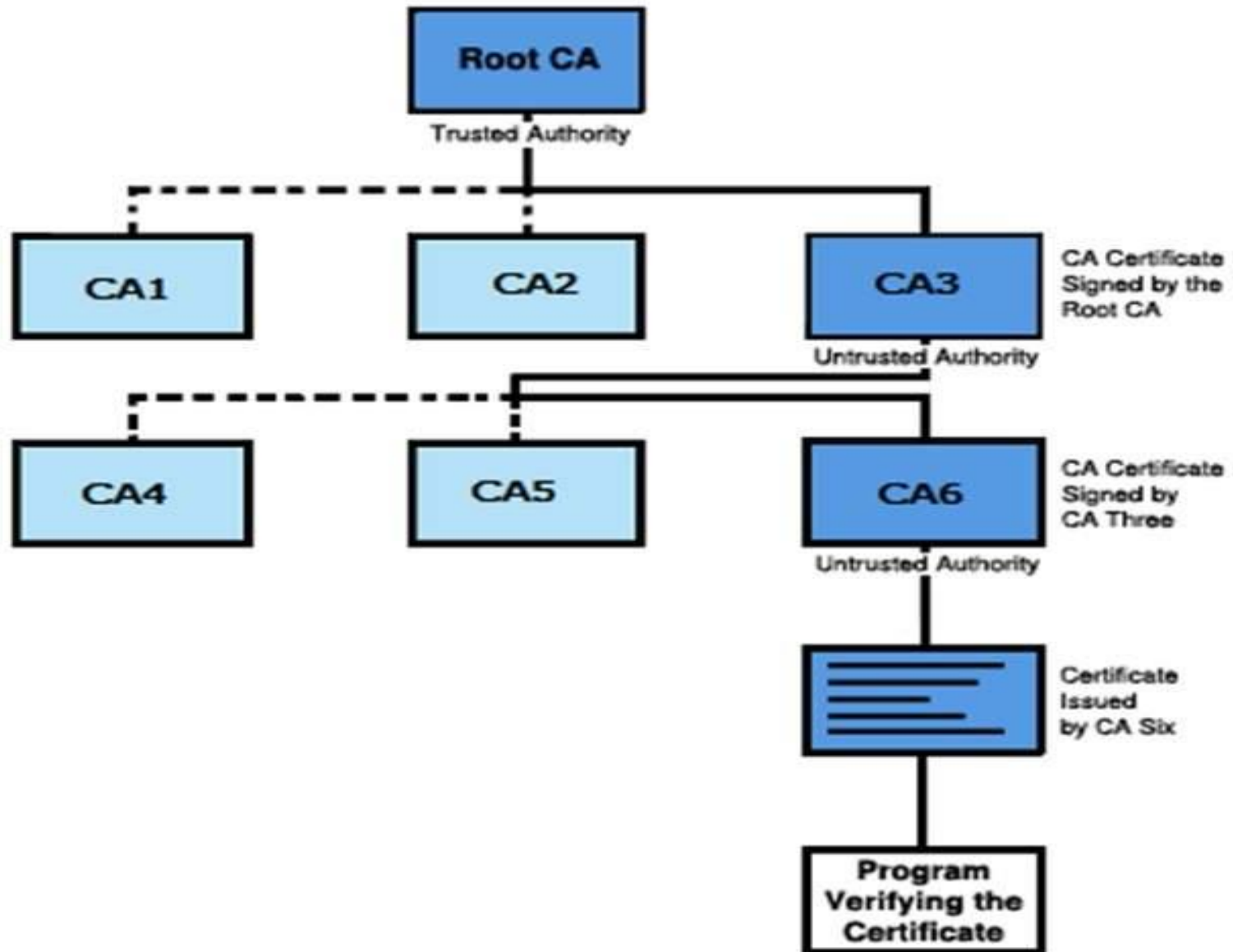


Certificate chain

- A certificate chain traces a path of certificates from a branch in the hierarchy to the root of the hierarchy.
- Example:

Suppose, a CA hierarchy with a certificate chain leading from an entity certificate through two subordinate CA certificates (CA6 and CA3) to the CA certificate for the root CA.

Certificate chain



- Verifying a certificate chain is the process of ensuring that a specific certificate chain is valid, correctly signed, and trustworthy.
 - A client whose authenticity is being verified supplies his certificate
 - Verifier takes the certificate and validates by using **public key of issuer** (found in the issuer's certificate which is in the chain next to client's certificate)

- Now if the higher CA who has signed the issuer's certificate, is trusted by the verifier, verification is successful
- Else, the issuer's certificate is verified in a similar manner as done for client in above steps.
- This process continues till either trusted CA is found in between or else it continues till Root CA.